Chapter 6: Orchestration

Preparation For AI: From Raw Data To Reliable Models

Alexander D'hoore

Orchestration

So far, you started most work yourself.

Chapter 6 asks:

```
How does platform work run repeatably?
```

The answer is **orchestration**.

From Commands To Workflows

Manual scripts are how you learn the pieces.

Orchestration is how a team runs them repeatably.

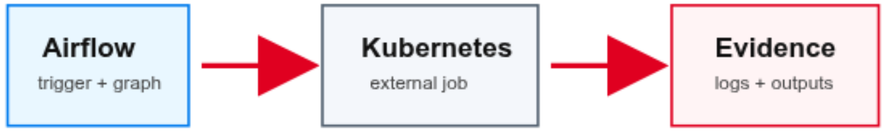
Chapter 6 moves from:

```
remembered commands
```

to:

```
visible platform workflows
```

An orchestrated workflow can be inspected, triggered, rerun, debugged, and discussed by the team.



Why Manual Runs Break

Manual workflows depend on memory:

- which command?
- which directory?
- which environment?
- which inputs?
- which output location?

That is fragile for shared data work.

Airflow, Kubernetes, Object Storage

In this chapter:

- Airflow controls the run
- Kubernetes does the compute
- object storage keeps the evidence

That separation matters.

Airflow should coordinate work, not become the dataframe engine.

DAGs And Tasks

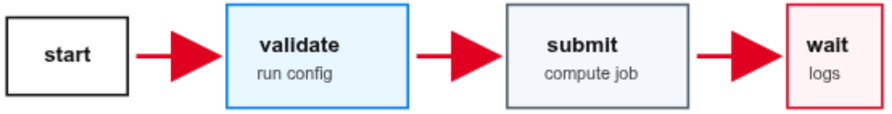
The core Airflow object is a DAG.

DAG means:

```
directed acyclic graph
```

In practical terms, it is a workflow definition.

Tasks are the steps. Arrows define order.



Keep DAGs Lightweight

Do not hide all transformation logic in the DAG file.

Better:

```
Airflow DAG
  -> calls clear task code
  -> task code reads and writes data
  -> Airflow records state and logs
```

The DAG answers what runs when.

The task code does the work.

Run Parameters

Parameters make one DAG reusable.

The same workflow can run for different:

- participants
- buckets
- months
- thresholds
- training settings

Manual Airflow runs receive those values as JSON.

Configuration JSON

participant: preparation-for-ai-XX
namespace: preparation-for-ai-XX
output_bucket: preparation-for-ai-XX
input_month: 2024-01
minimum_rows: 100000

External Compute Pattern

Airflow does three operational jobs:

- starts external compute
- waits for success or failure
- records logs and run state

The task code does the data or model work.

Logs Are Evidence

In orchestration, logs are not noise.

They prove:

- which configuration was used
- which external job was created
- whether it succeeded
- where outputs were written

Logs answer: **what happened during this run**

Quality Gates Belong In Workflows

A check that runs only from memory is weak.

A check in the DAG is stronger.

Examples:

- manifest exists
- row count is high enough
- expected columns exist
- splits do not overlap
- validation error is acceptable

Failure Should Be Visible

Failing early is a feature.

The workflow should stop before bad data moves forward.

Good failure tells you:

- which task failed
- why it failed
- which inputs were used
- where to look next

Idempotent Tasks

Airflow can retry failed tasks.

Retries are safe only when tasks are designed carefully.

Idempotent means:

```
same inputs -> same intended result
```

Run-specific output prefixes are a simple habit that helps.

Run-specific output prefix

outputs/chapter06/taxi-python/
run_id=manual-example/

keeps reruns separate
makes evidence easier to find

Schedules And Backfills

Airflow DAGs can run manually or on a schedule.

Schedules are useful when data arrives predictably.

Backfills rerun historical intervals after:

- pipeline failures
- late data arrivals
- transformation bug fixes

They are risky if tasks are not idempotent.

Lab Orientation

There are two workflows:

```
taxi analytics Python job
video GPU training job
```

Airflow coordinates both.

The compute changes. The orchestration patter
stays the same.

Taxi Workflow

The first workflow:

```
Airflow -> Kubernetes Job -> Python
```

The Python job reads shared object storage.

It writes Silver and Gold outputs to the participant bucket.

Students connect the Airflow run to object-storage evidence.

Video GPU Workflow

The second workflow:

```
Airflow -> Kubernetes GPU Job -> PyTorch
```

The job reads video clips and a manifest from object storage.

It writes model evidence back to the participant bucket.

Queueing is expected when GPU capacity is shared.

The Main Pattern

Across both labs:

```
trigger
  -> validate parameters
  -> submit external compute
  -> wait and record logs
  -> inspect object-storage evidence
```

This is the operational habit.

What To Notice In The Lab

For every run, connect:

- Airflow DAG run
- task logs
- Kubernetes job or pod
- object-storage output prefix
- downloaded evidence

If those links are clear, the workflow is inspectable.

Orchestration

Orchestration turns scripts into repeatable workflows.

Airflow coordinates the run.

Kubernetes executes the work.

Object storage keeps the evidence.

The question to carry forward:

```
Where would I look if this failed tomorrow morning?
```